KLEENE'S THEOREM

CTND...

PROOF PART 3

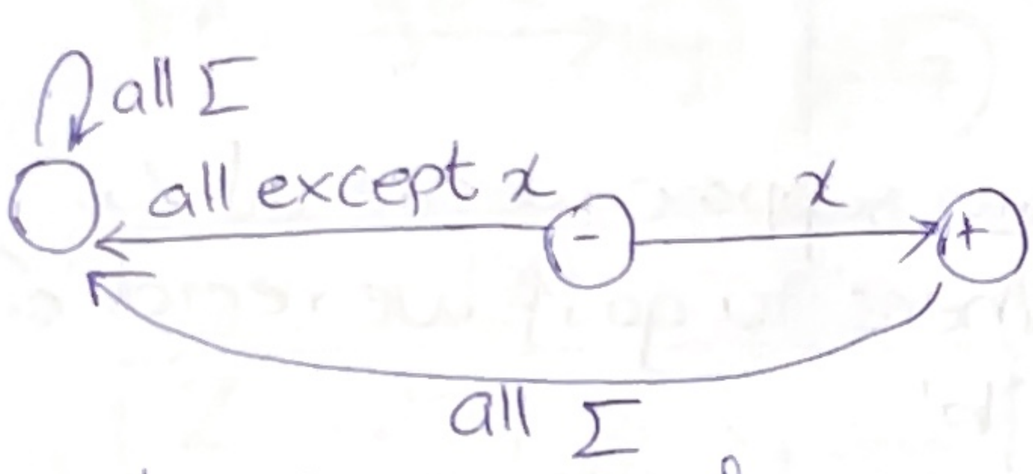
Converting Regular Expression(s) into Finite Automata.

Rule no.1

There is a finite automata that accepts any particular letter of the alphabet.
There is a finite automata that only accepts the word $\wedge$.

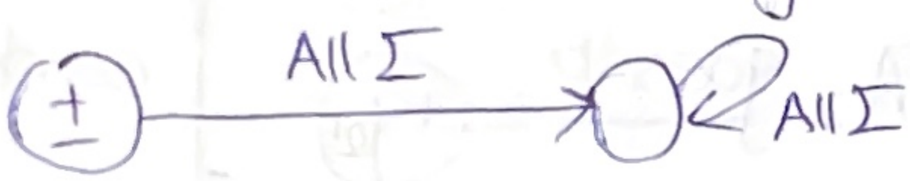
Proof:

* if x is in Σ then FA;



accepts only the word x.

* One FA that accepts only $\wedge$ is;

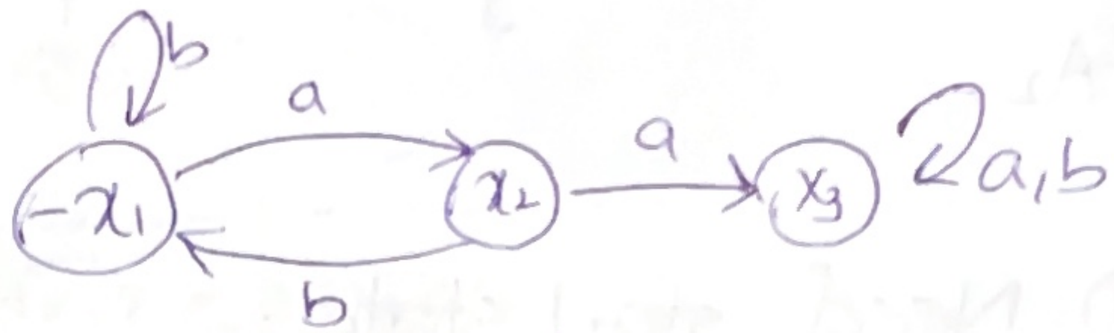


ROLE NO.2

If FA_1 accepts language defined by r_1 (regular expression) and FA_2 accepts language defined by RE r_2 than $FA_3 = (r_1 + r_2)$

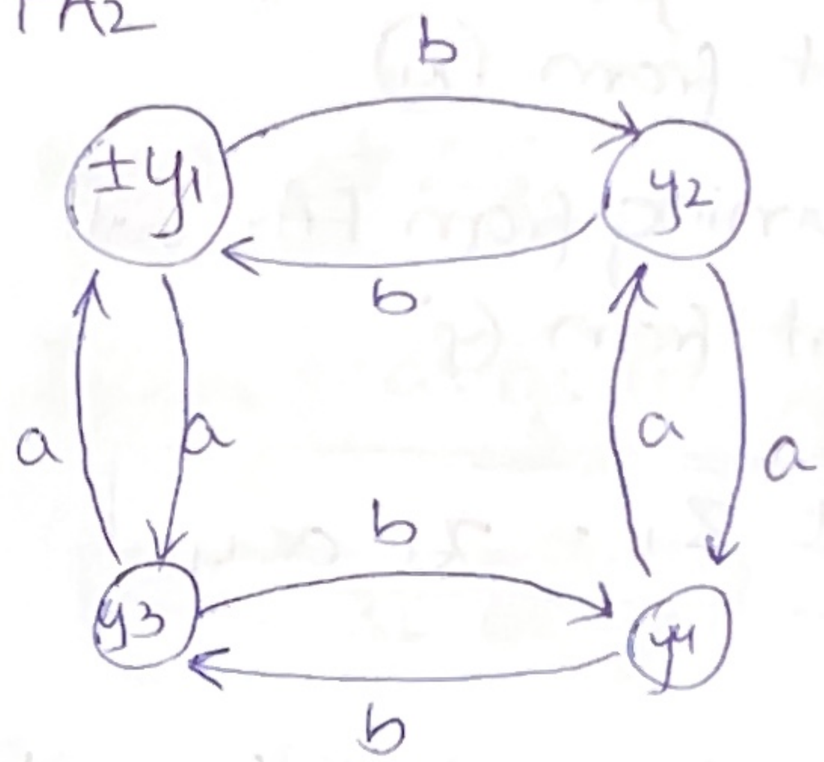
Proof:

FA_1



	a	b
$-x_1$	x_2	x_1
x_2	x_3	x_1
$+x_3$	x_3	x_3

FA_2



	a	b
$\pm y_1$	y_3	y_2
y_2	y_4	y_1
y_3	y_1	y_4
y_4	y_2	y_3

$FA_3 \Rightarrow$ will accept the union of these two languages.

Call its states as Z_1, Z_2, Z_3

* we will define the transition table for this machine as well.

While building the new FA we will keep track of where the input would be if it is running on FA_1 & FA_2.

① Need start state :-

- Must combine start state from FA_1 & FA_2.

call (Z1)

- if running from FA_1, will start from (x1)
- if runing from FA_2, will start from (y1)

$\therefore$ $\boxed{\pm Z_1 = x_1 \text{ or } y_1}$

→ what's the new state that can occur if we read 'a' or 'b' at (Z1)?

'If we read a'

FA_1 goto (x2)
FA_2 go to (y3)
] New combination

call this as; (Z2)

$\boxed{Z_2 = X_2 \text{ or } y_3}$

Similarly
If we read 'b' at Z_1

FA_1 goto (x1)
FA_2 goto (y2)
] New combination

call this as (Z3)

$\boxed{Z_3 = X_1 \text{ or } y_2}$

Begining of Transition table

	a	b
$\pm Z_1$	Z_2	Z_3

Now suppose we are at Z_2
Where to go if we read 'a' or 'b'
if we read 'a'

FA_1 goes to (+x3)
FA_2 goes to (+y1)
] New combination.

call this as (Z4)

$\boxed{+Z_4 = X_3 \text{ or } y_1}$

if read 'b'

FA_1 goto (Z1)
FA_2 go to (y4)

call this $\boxed{Z_5 \Rightarrow X_1 \text{ or } y_4}$

Updated Transition table.

	a	b
±Z_1	Z_2	Z_3
Z_2	Z_4	Z_5

Next at Z_3

Where to go if read 'a'

FA_1 goes to X_2
FA_2 goes to y_4
] call it Z_6

Where to go if read 'b'

FA_1 go to X_1
FA_2 go to $+y_1$
] call it $+Z_7$

$\therefore$ $Z_6 = X_2$ OR y_4

$Z_1 \Rightarrow +Z_1 \Rightarrow X_1$ or y_1

Updated Transition table.

	a	b
±Z_1	Z_2	Z_3
Z_2	Z_4	Z_5
Z_3	Z_6	Z_1

Next at Z_4

If FA_1 read a goes to $+X_3$
FA_2 read a goest y_3
Call it Z_7

$+Z_7 = +X_3$ or y_3

If FA_{01} read b goes to $+X_3$
FA_2 read b goes to y_2

$+Z_8 = +X_3$ or y_2

Next at Z_5

If read 'a' go to X_2 or y_2

$Z_9 = X_2$ or y_2

If read 'b' go to X_1 or y_3

$Z_{10} = X_1$ or y_3

Next at Z_6

If read a $\longrightarrow +X_3$ or y_2.

$+Z_8 = X_3$ or y_2

If read b $\longrightarrow X_1$ or y_3

$Z_{10} = X_1$ or y_3

Next Z_7

If read 'a' $\longrightarrow$ x_3 or $y_1 \Rightarrow Z_4$

If read 'b' $\longrightarrow$ x_3 or y_4.

$+Z_{11} = x_3$ or y_4

Next Z_8

If read 'a' $\longrightarrow$ x_3 or $y_4 \Rightarrow +Z_{11}$

If read 'b' $\longrightarrow$ x_3 or $y_1 \Rightarrow Z_4$

Next Z_9

If read 'a' $\longrightarrow$ x_3 or $y_4 \Rightarrow +Z_{11}$

If read 'b' $\longrightarrow$ x_1 or $y_1 \Rightarrow Z_1$

Next Z_{10}

If read 'a' $\longrightarrow$ x_2 or y_1 Z_{12}

If read 'b' $\longrightarrow$ x_1 or y_4 Z_5

$Z_{12} = x_2$ or y_1

Next Z_{11}

If read 'a' $\longrightarrow$ x_3 or $y_1 \Rightarrow Z_8$

If read 'b' $\longrightarrow$ x_3 or y_3 Z_7

Next Z_{12}

If read a $\xrightarrow{\text{goto}}$ x_3 or y_3

$Z_7 = x_3$ or y_3

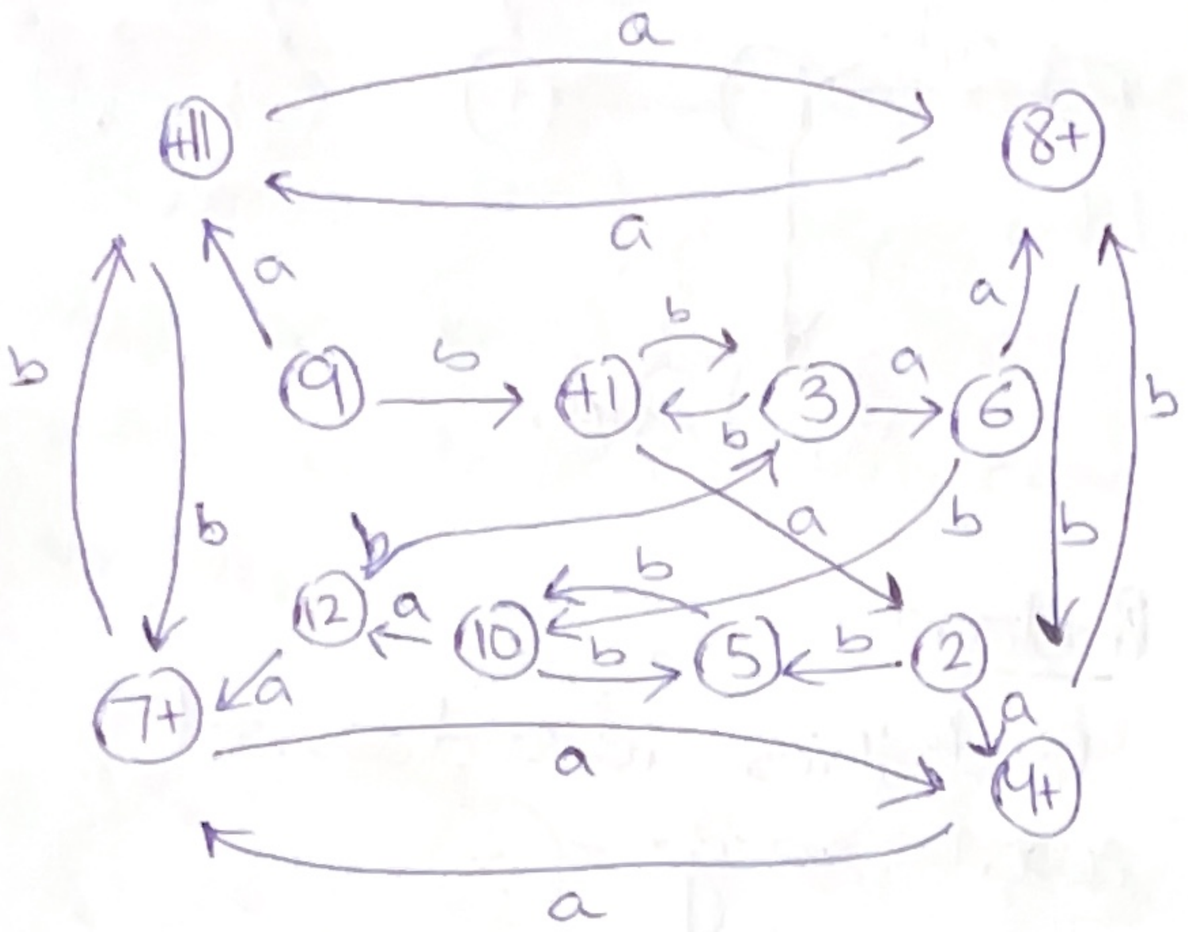
If read b $\xrightarrow{\text{goto}}$ x_1 or y_2

$Z_3 = x_1$ or y_2

Complete transition table.

	a	b
$+Z_1$	Z_2	Z_3
Z_2	Z_4	Z_5
Z_3	Z_6	Z_1
Z_4	Z_7	Z_8
Z_5	Z_9	Z_{10}
Z_6	Z_8	Z_{10}
Z_7	Z_4	Z_{11}
Z_8	Z_{11}	Z_4
Z_9	Z_{11}	Z_1
Z_{10}	Z_{12}	Z_5
Z_{11}	Z_8	Z_7
Z_{12}	Z_7	Z_3

part simultaneously.



SUMMARIZING ALGORITHM:- Steps.

What did we do.

① started with FAs with states x_1, x_2, x_3 and y_1, y_2, y_3

② Build a New FA with states z_1, z_2, z_3 where each z is form with x something and y something.

③ combination of state xstart or ystart is the start state of new FA.

- If either x part or y part is final then the corresponding Z is final.

- To go from one z to another by reading a letter from the input string
  ↳ we see what happens to x & y part simultaneously.

formula

$\sim x \sim$

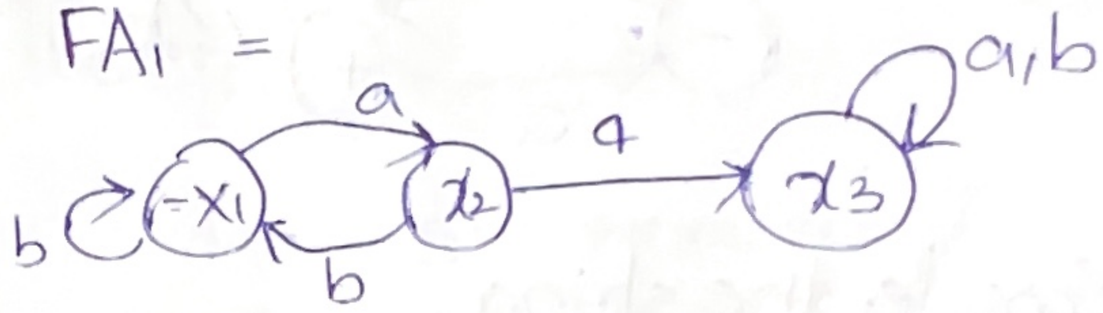
Z new after letter P =

[x new after letter P on FA_1] or

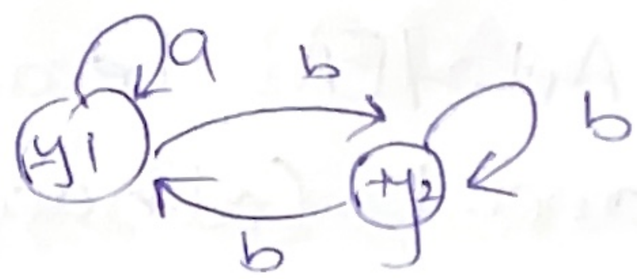
[y new after letter P on FA_2].

EXAMPLE FOR STUDENTS

FA_1 =



FA_2 =



Ask them to do as class work.

STUDENT HOMEWORK

Example pg # 114, 115

Rule no. 3

$\sim x \sim$

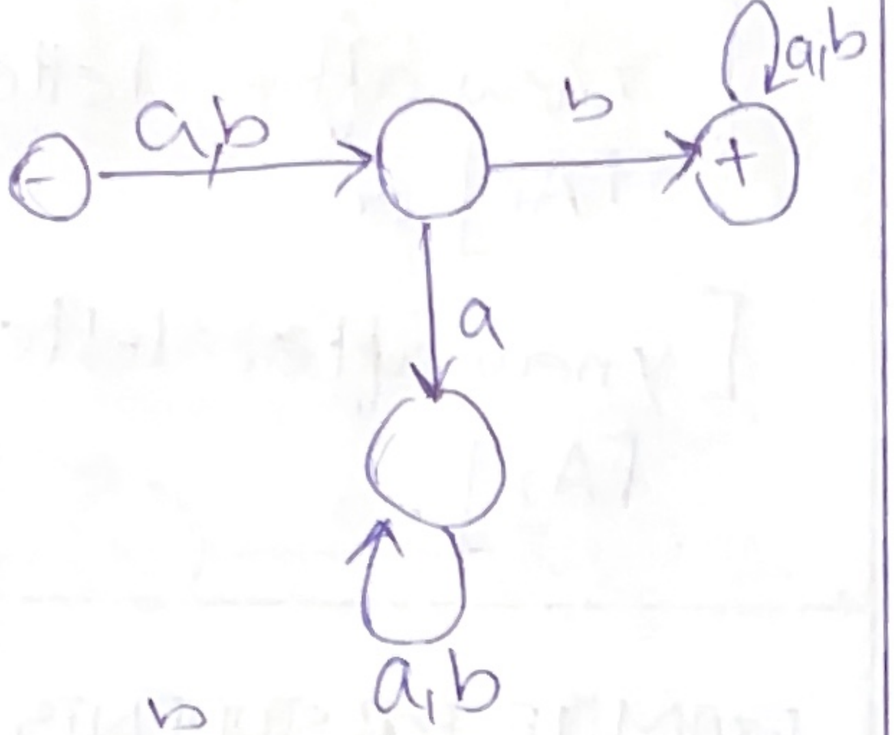
if FA_1 accepst language r_1

AND FA_2 accepts language r_2.

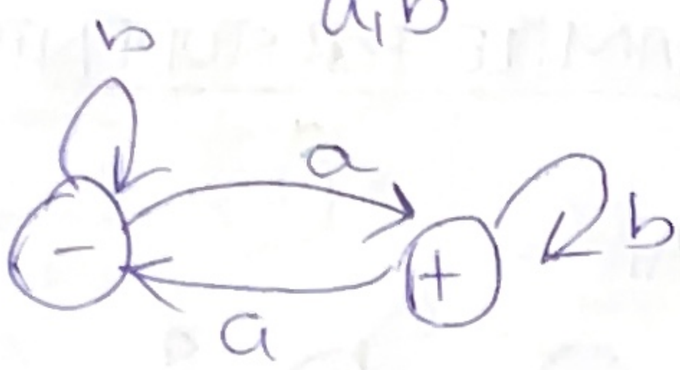
then FA_3 is defined as $FA_3 = r_1 r_2$

Proof :- Verify this with constructive algorithm.

FA_1 =



FA_2 =



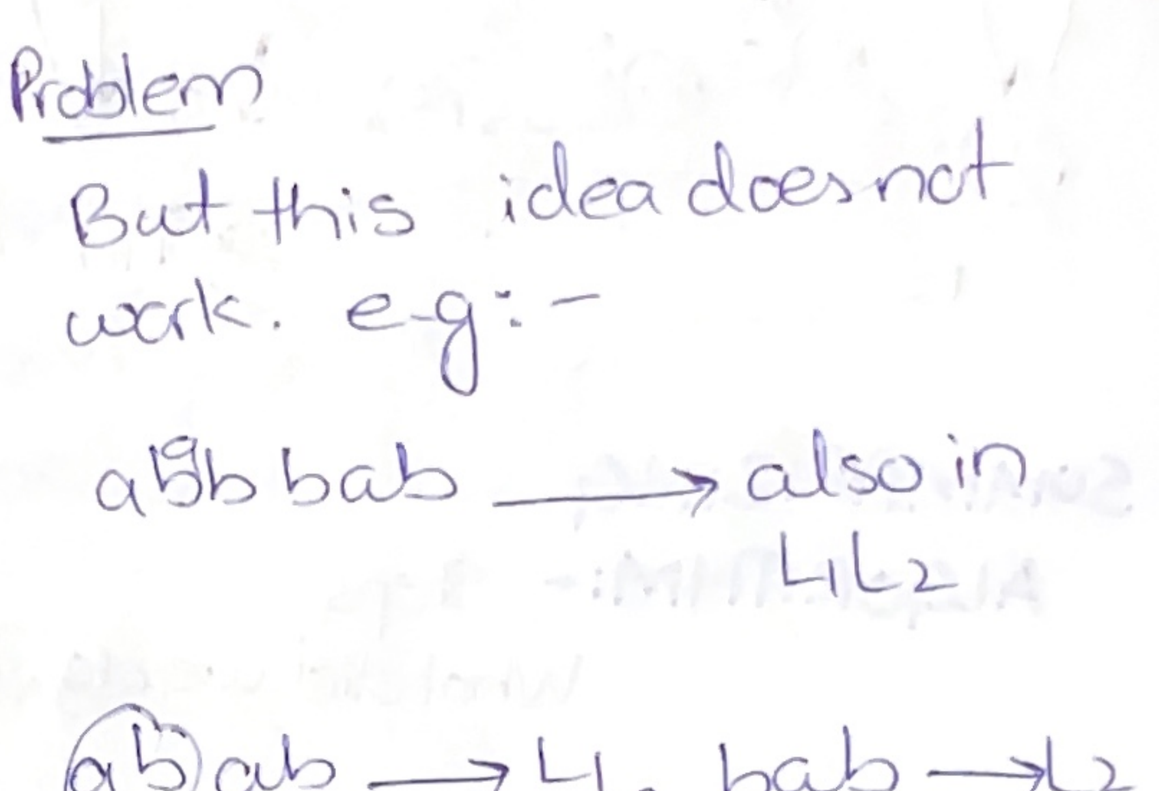
Consider the string.

'ababbaa' is a word in $L_1/FA_1 . L_2/FA_2$ because $L_1(ab)$ and $L_2(abbaa)$

Begin with FA_1 and we will reach (+) after the second letter.

Next Jump over to FA_2 and begin running what is left.

The following is what we want to build



Problem

But this idea does not work. e.g:-

abbbab bab ⟶ also in $L_1 L_2$

(ab) ab ⟶ L_1, bab ⟶ L_2

takes us to the final state (+) of FA_1 and we cannot say that we are done with FA_1

* So, How do we know when to jump from FA_1 to FA_2?

with ababbaa

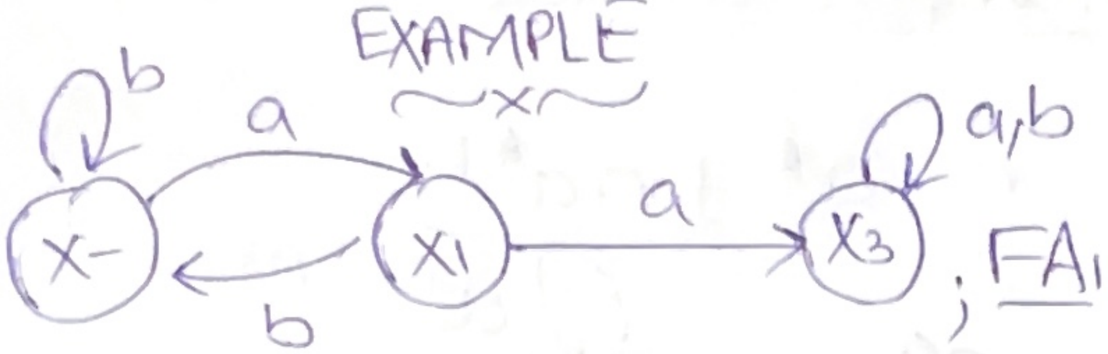
⟶ we can jump after reaching (+)

with ababbab

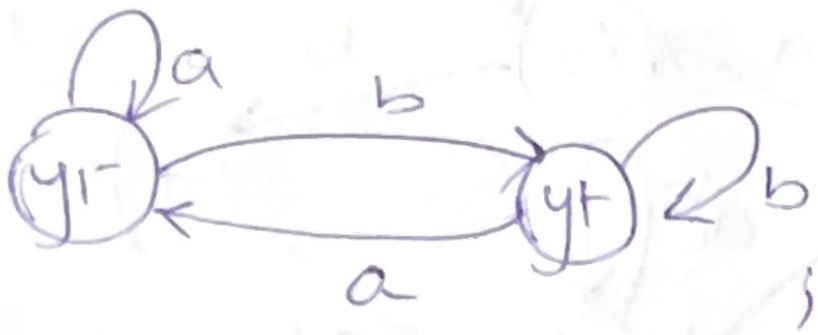
⟶ we can only jump after looping several times on final state of FA_1

.We must build a machine with characteristics of starting out like FA_1 and following along until it reaches final state were either continue on FA_1 or jump to FA_2.

EXAMPLE



; FA_1



; FA_2

$L_1L_2 = r_1r_2$.

$\rightarrow Z_1 = X_1$

means input running on FA_1

Read a

$X_2 \longrightarrow Z_2$

Read b

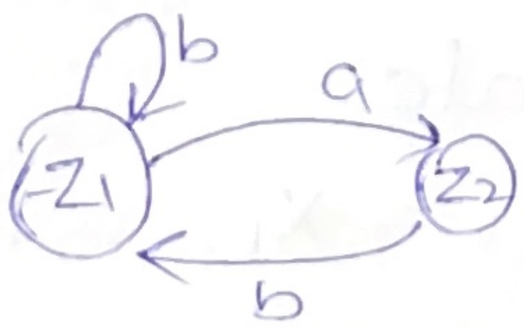
$X_{01} \longrightarrow Z_1$

So;

$Z_1 = X_1$

$Z_2 = X_2$.

$FA_3 =$



$\rightarrow Z_2 = X_2$.

Read a

$X_3 \longrightarrow Z_3$

Read b

$X_1 \longrightarrow Z_1$

-means either we reached final state of FA_1 and are done for the first half of input string and jump to y_1.

-OR we continue over FA_1

Therefore Z_3

$\begin{cases} X_3 - \text{still running on } FA_1 \\ \text{or} \\ y_1 - \text{have begun to run on } FA_2. \end{cases}$

There are three possible interpretations.

① back on x_3 and continuing to run the string on FA_1

② just finished FA_1 and now y_1 to begun with FA_2.

③ have loope back from y_1 to y_2 running already on FA_2.

$\boxed{Z_3 = X_3 \text{ or } y_1}$

Next on Z_3 then

Read a

$FA_1 \longrightarrow X_3 \text{ or } y_1$

$FA_2 \longrightarrow y_1$

i.e $Z_3 = X_3 \text{ or } y_1 \text{ or } y_1$.

Read b.

$FA1 \longrightarrow X_3 \text{ or } y_1$

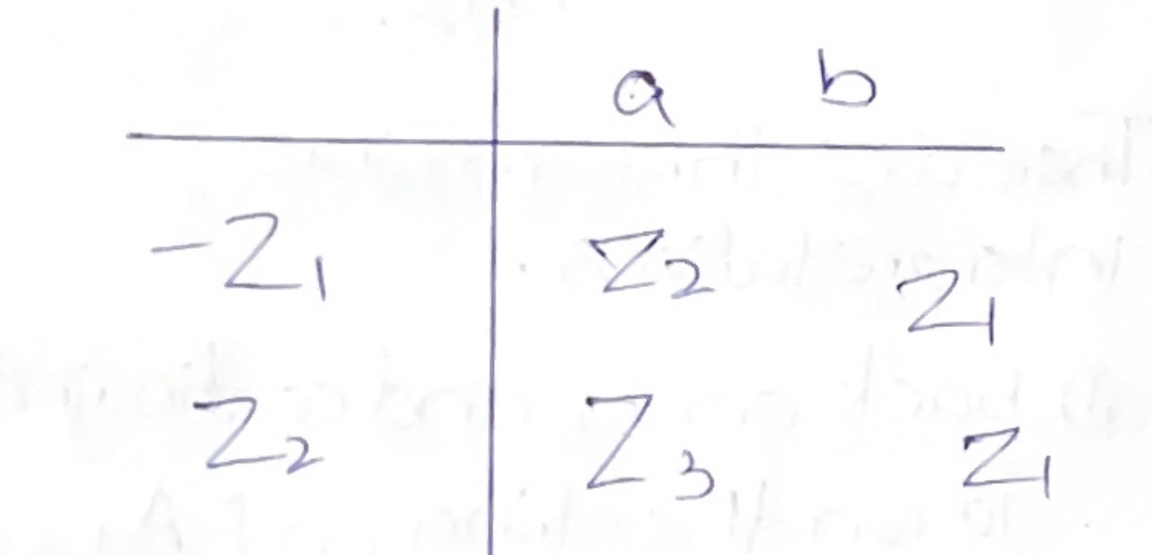
$FA_2 \longrightarrow y_2$

i.e

$+Z_4 = X_3 \text{ or } y_1 \text{ or } y_2$.

hence now completeing the transition table accordingly.

	a	b
$\to Z_1$	Z_2	Z_1
Z_2	Z_3	Z_1
$Z_3(X_3 \text{ or } y_1)$	Z_3	Z_4
$\pm Z_4(X_3 \text{ or } y_1 \text{ or } y_2)$	Z_3	Z_4



RULE NO. 4

~x~

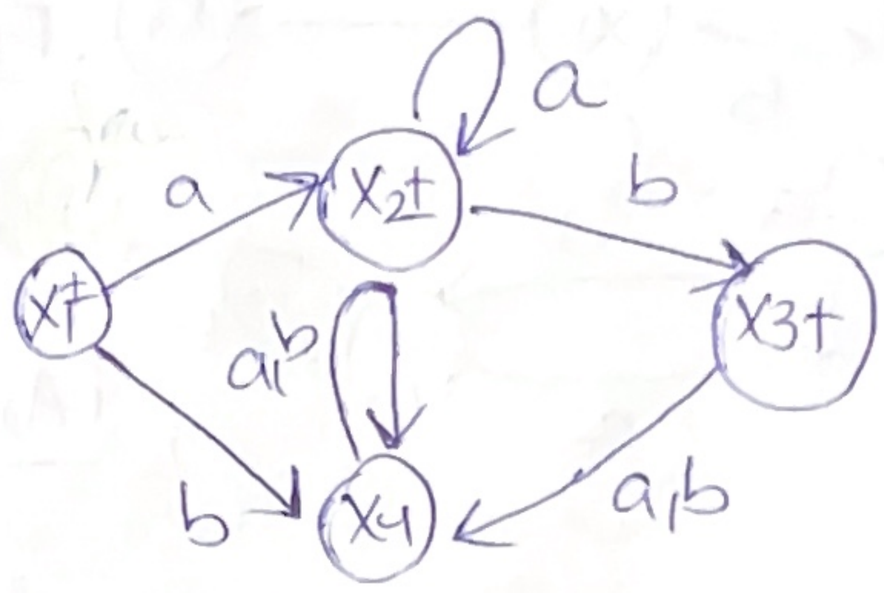
If r is a R.E

AND FA_1 accepts r_1

then FA_2 accepts r_2^*

Example; Consider

$r = a^* + aa^*b$



Note: X_4 is a reject/Dead state.

- Λ is also acceptable.

$r^* \longrightarrow$ is all the strings without a double 'b' that donot begin with b.

i.e aba, abaabaaa are not included.

Now new machine FA_2

start state.

$\pm Z_1 = X_1$

At (+Z1)

Read a

$+X_2$ or $+X_1 = +Z_3$.

Read b

$X_4 = Z_2$

At (Z_2)

Read a

$X_4 = Z_2$

Read b

$X_4 = Z_2$

At (Z_3)

Read a

X_2 or X_1 or $X_2 = +Z_3$

Read b

$+X_3$ or X_4 or $X_1 = Z_4$

At (+Z4)

Read b

$X_4 \Rightarrow Z_2$

Read a

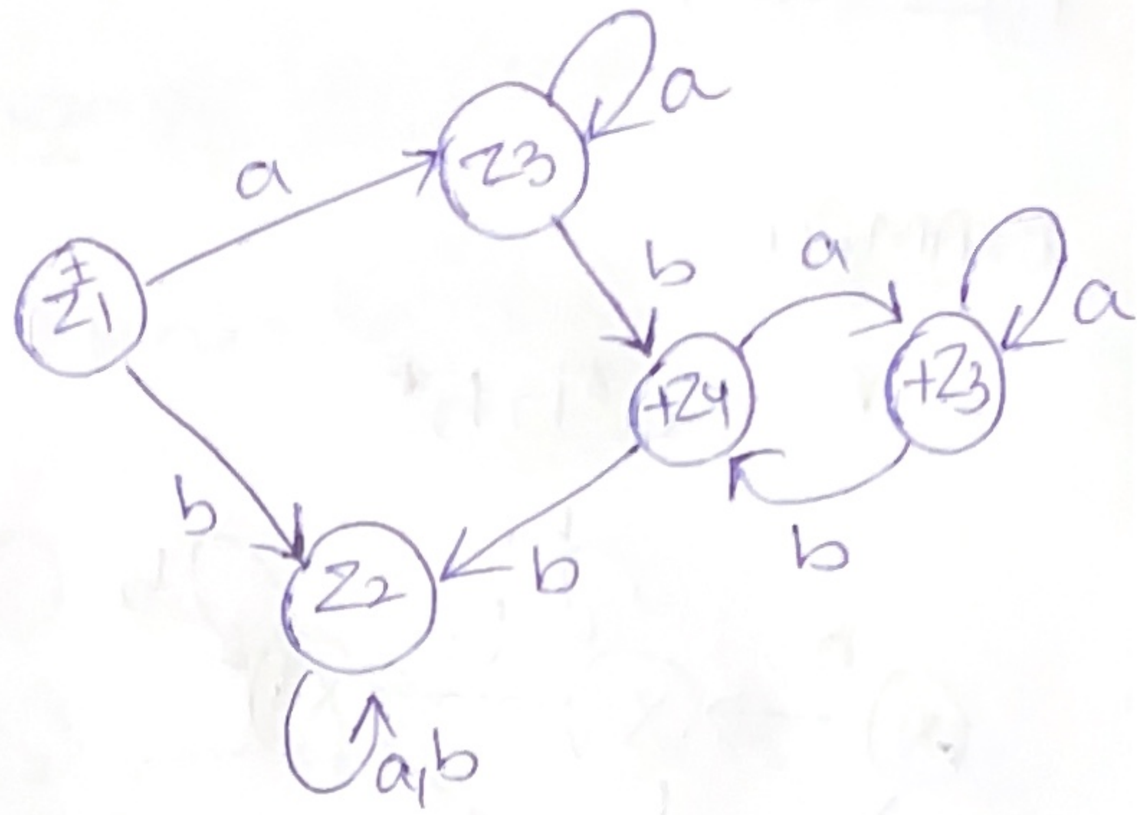
X_1 or X_2 or $X_4 \Rightarrow +Z_5$

At (+Z5)

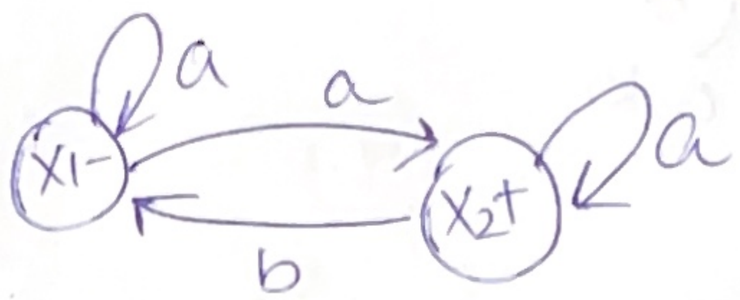
Read a

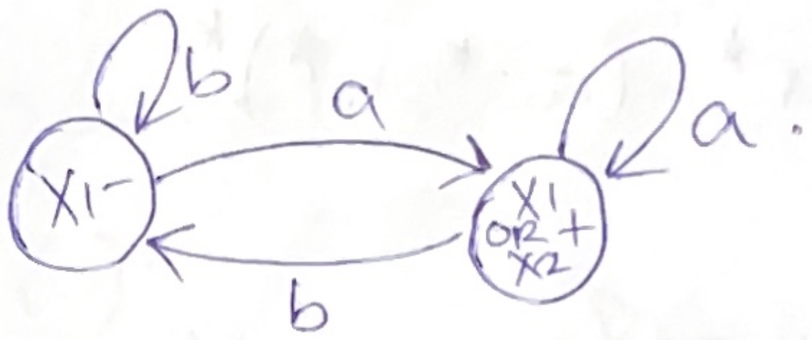
X_1 or X_2 or $X_4 = Z_5$

Read b X_4 or X_3 or $X_1 \Rightarrow Z_4$



Example

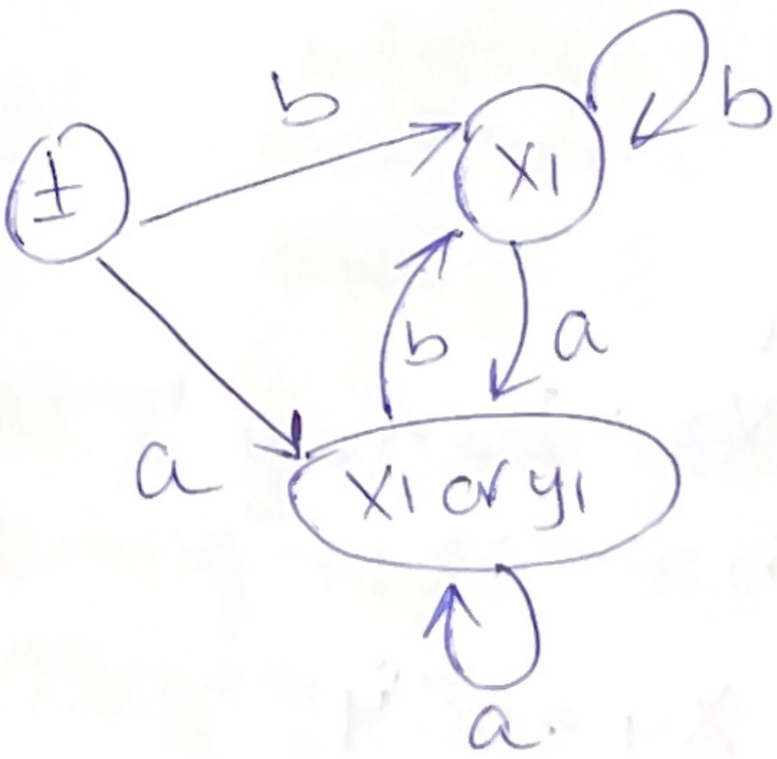




∧ ?

Begin with special case (±)

Remember Rule 1 & Rule ___.

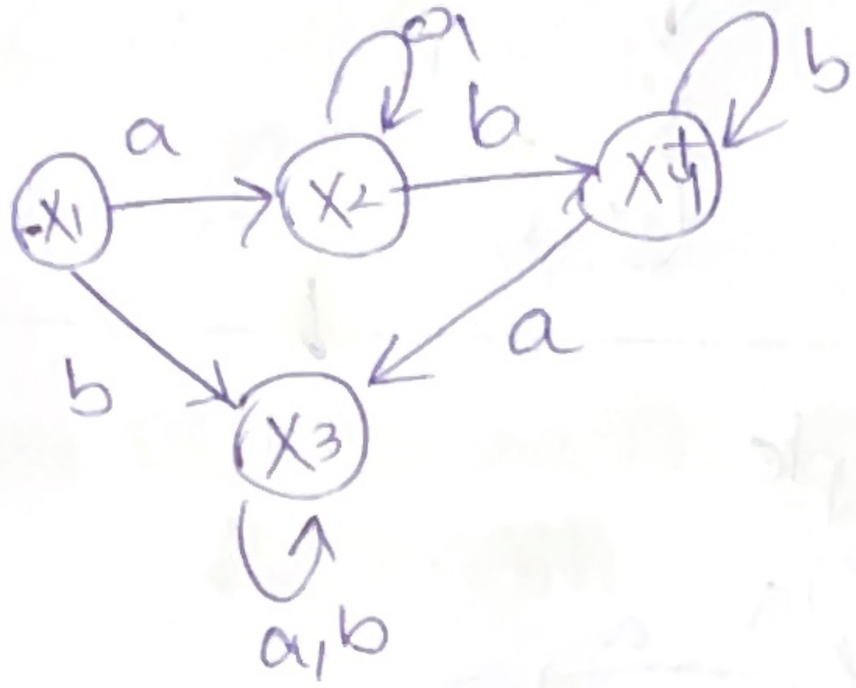


P.T.O →

Pgno. Kalten)

EXAMPLE :

$$r = aa^*bb^*$$



Now for

$$r^* = (aa^*bb^*)^*$$

let $-Z_1 = X_1$

Read a

$\rightarrow X_2 = Z_2$

Read b

$\rightarrow X_3 = Z_3$.

At Z_2

Read a

$\rightarrow X_2 = Z_2$

Read b

$\rightarrow X_4 = +Z_4$?

or X_1

At Z_3

Read a

$\rightarrow X_3 = Z_3$

Read b

$\rightarrow X_3 = Z_3$

At Z_4

Read a

X_3 or $X_2 = Z_5$

Read b

X_4 or X_3 or $X_3 = Z_6$

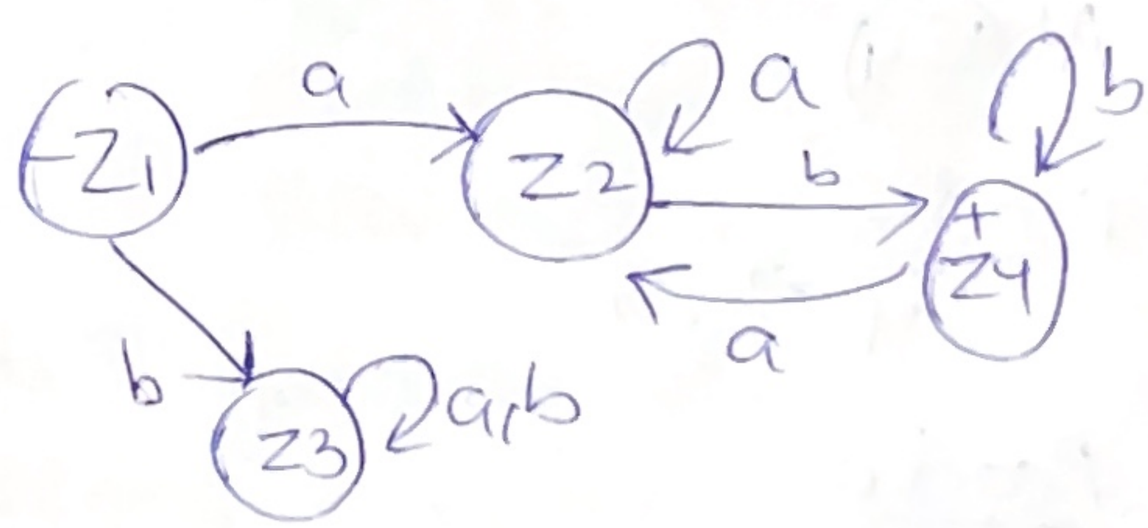
$\sim x \sim$

At Z_5 And similarly

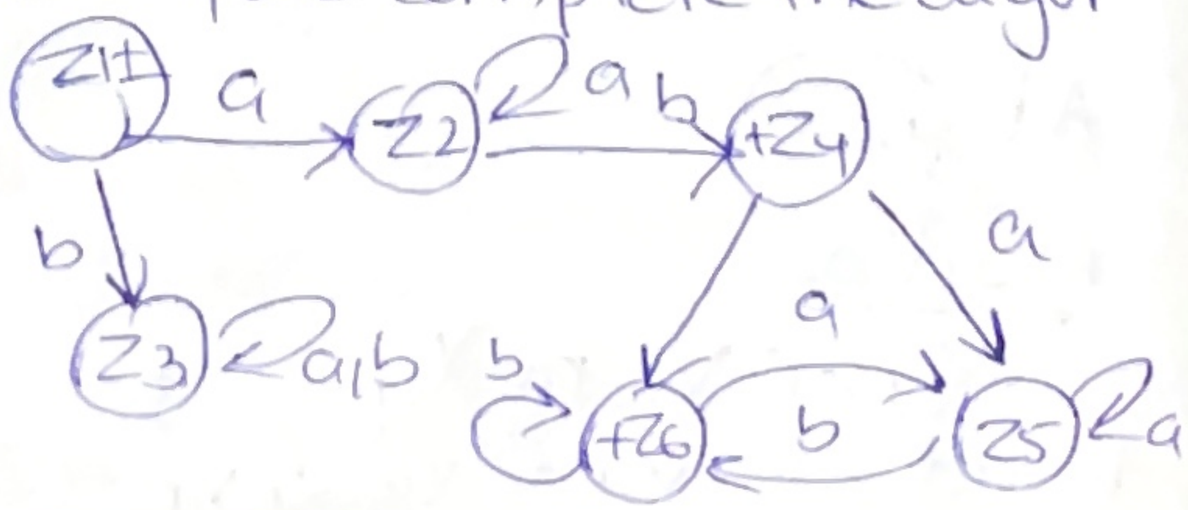
Read a complete for Z_5 & Z_6.

Read b

$\sim x \sim$



But if we ~~complete~~ the algorithm



- For final state
  * Compare example with previous one.

Note: Different and final state → same → edge recentering.

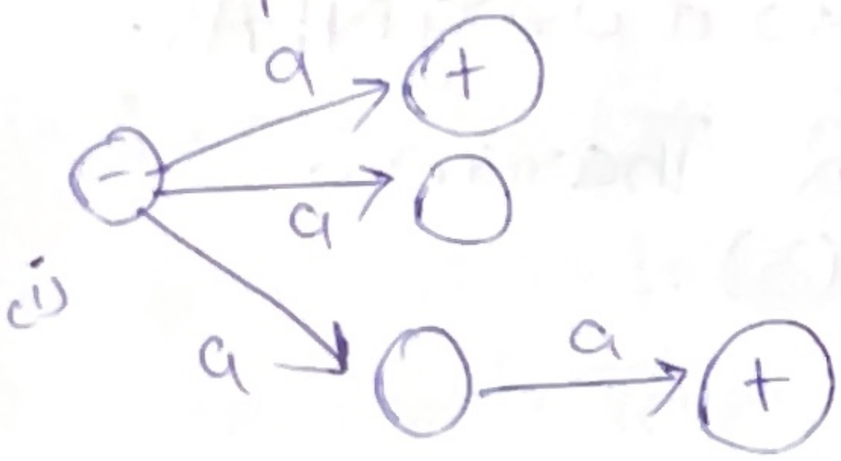
NON DETERIMINISTIC FINITE AUTOMATA.

Definition:- (NFA).

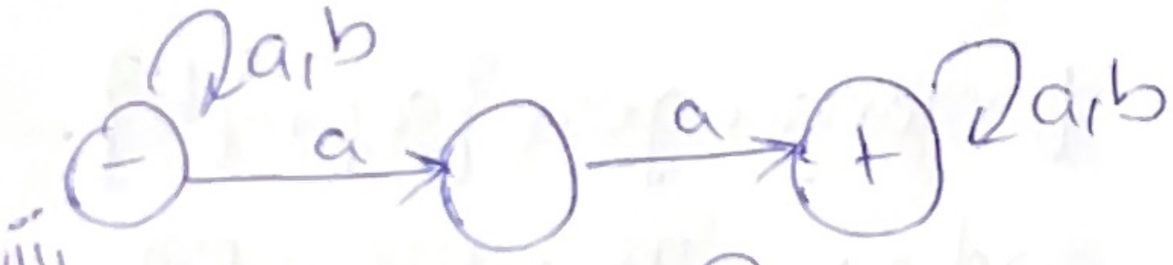
- It is a TG.
- Unique startstate.
* - Each edge labels are single alphabet.

example:-

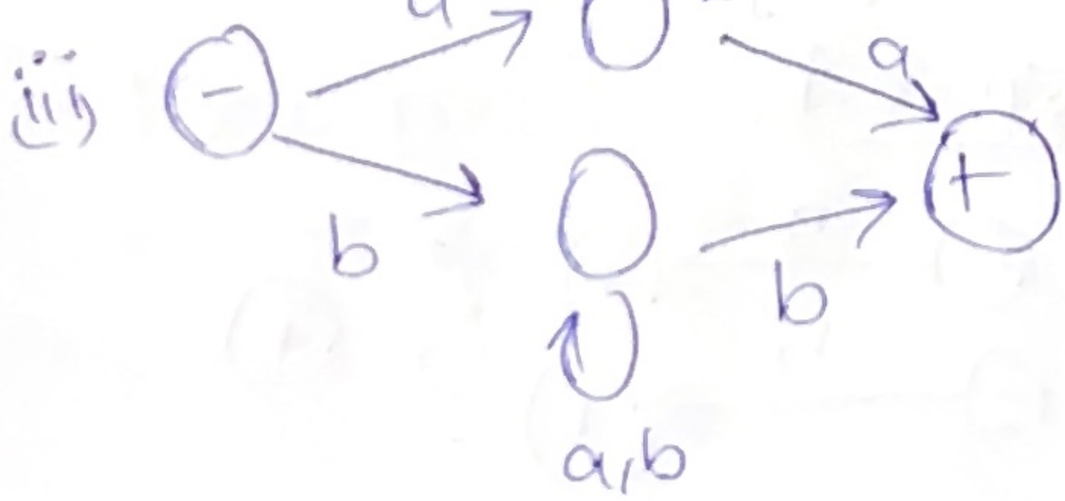
(i)



(ii)

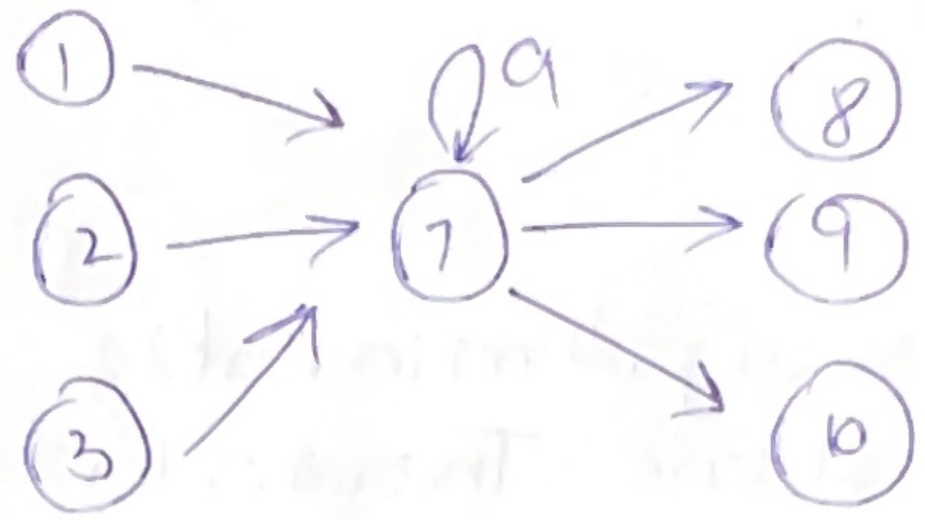


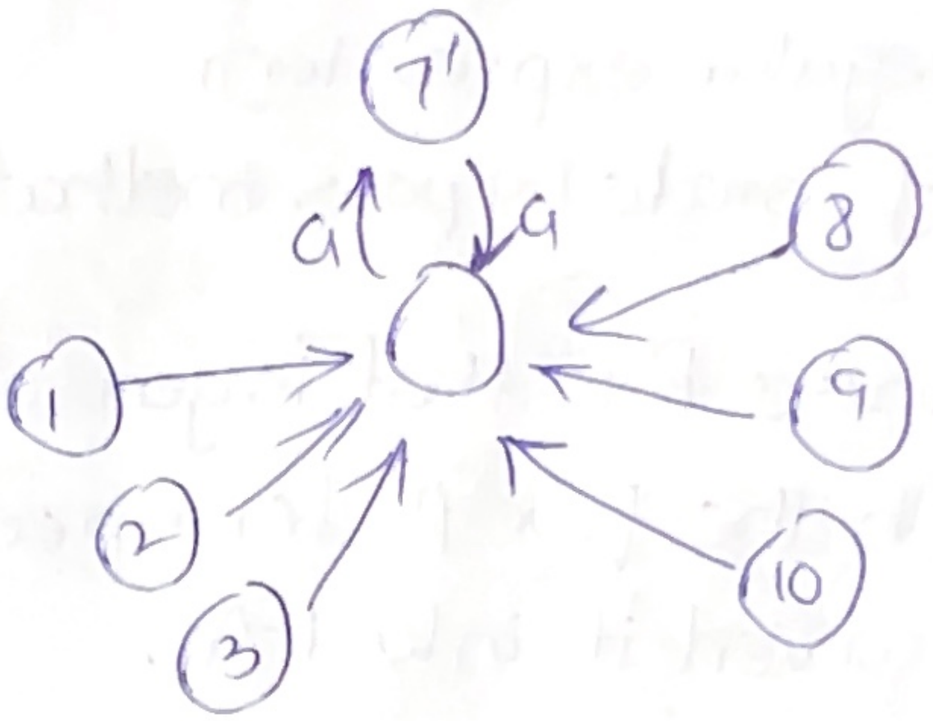
(iii)



Example Use of NFA

- Eliminate loop states.





ADVANTAGE

can record lop i-e whether path input follows goes through 7' or not.

At 7' you may set a flag altering one to the incidence of looping.

As Every TG can be converted to NFA

NFA is a TG.

So far, for every NFA there is some FA, that accepts exactly the same language as per the theorem 7 of book.

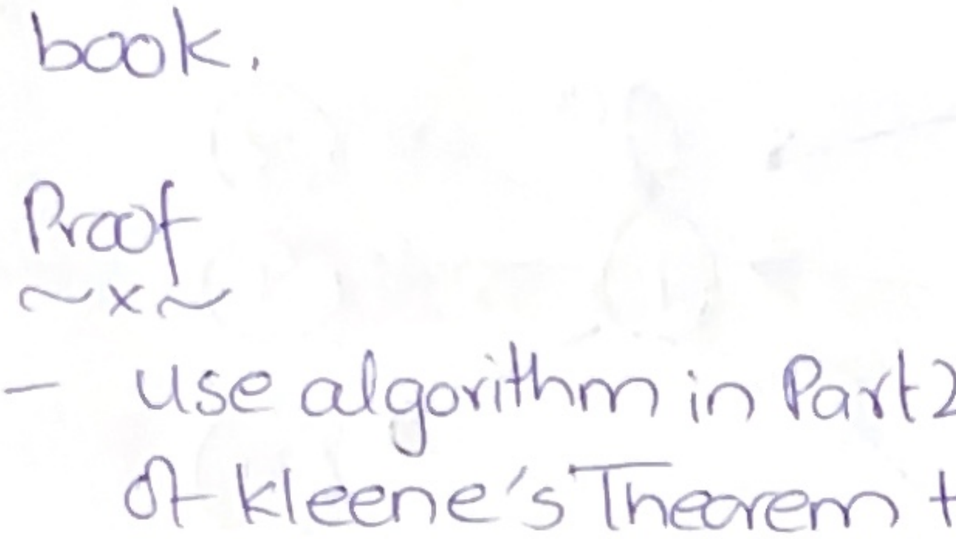
Proof

- use algorithm in Part 2 of Kleene's Theorem to convert the NFA to regular expression by state by pass method.

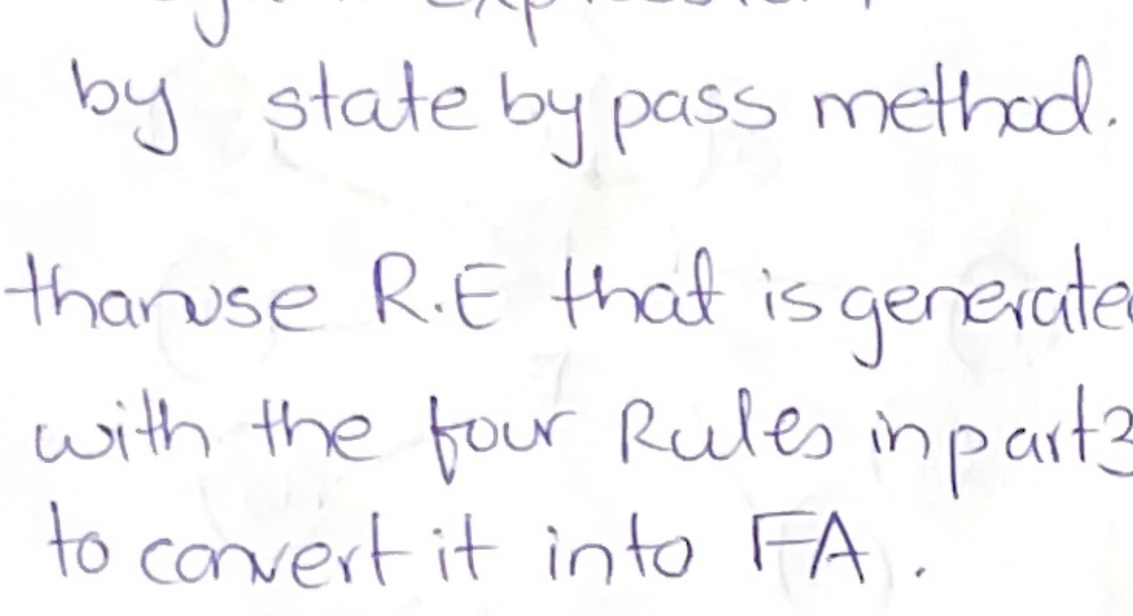
- thanuse R.E that is generated with the four Rules in part 3 to convert it into FA.

* Algorithm at pg # 138 chapter no. 7

EXAMPLE

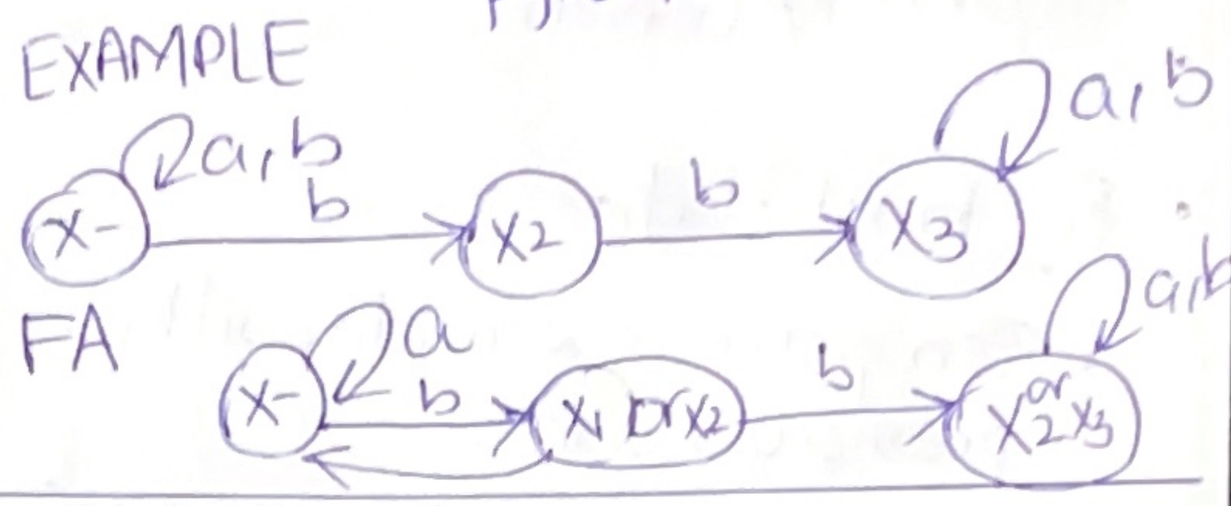


To; FA



Null calliebion should always have a self loop of a, b.

EXAMPLE



HOMEWORK

Chapter no. 7
page # 139
Examples.

NFAs & KLEENE THEOREM.

In kleen theorem we gave proof of.

$FA_1 + FA_2$, $FA_1 FA_2$ & FA_1^*

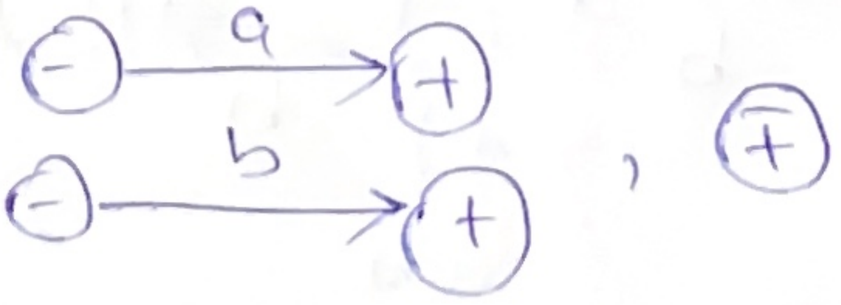
* lets do it using NFAs.

→ Proof 2. Theorem 6 part (3).

RULE 1

for languages {a}, {b}. and {Λ}. There are FAs.

step no. 1



Step no. 2

*Previously we studied that for every NFA there exists a FA.

Proof 2, Theorem 6 Part 3

RULE NO. 2

~x~

Given FA_1 and FA_2

Construct a union Machine $FA_1 + FA_2$.

Step no. 1.

introduce a new unique start state.

- Two outgoing a-edges & b-edges from the created start state respectively.

- Remove $\ominus$ signs from other start states.

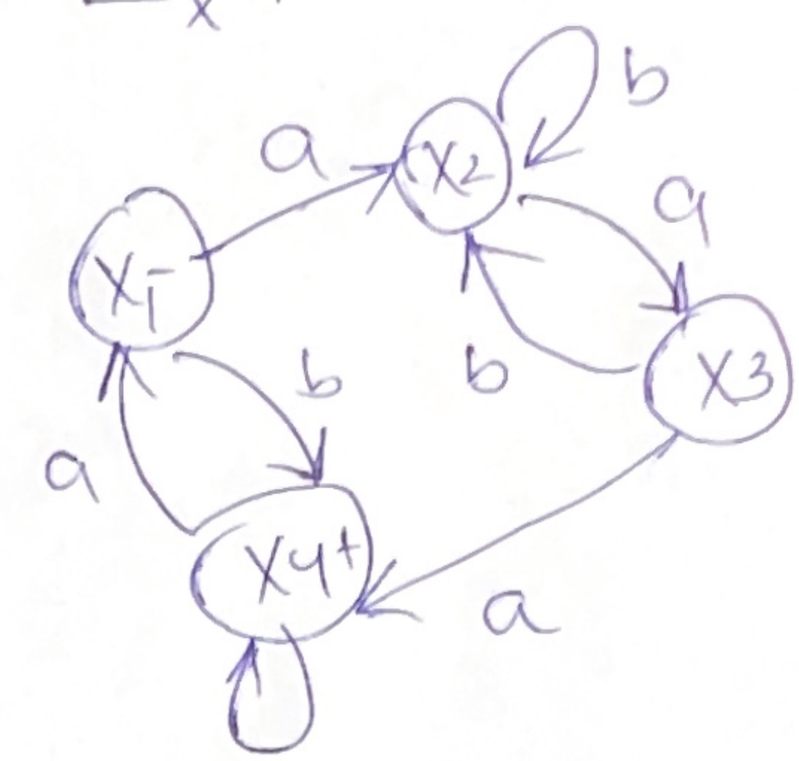
- New machine of as NFA will be generated.

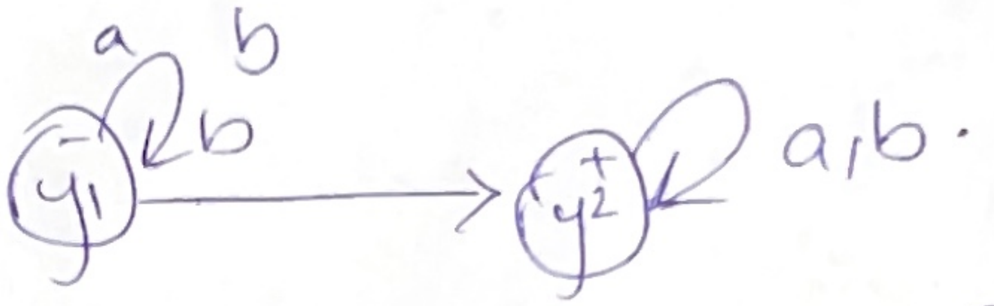
Step no. 2

Convert NFA to FA

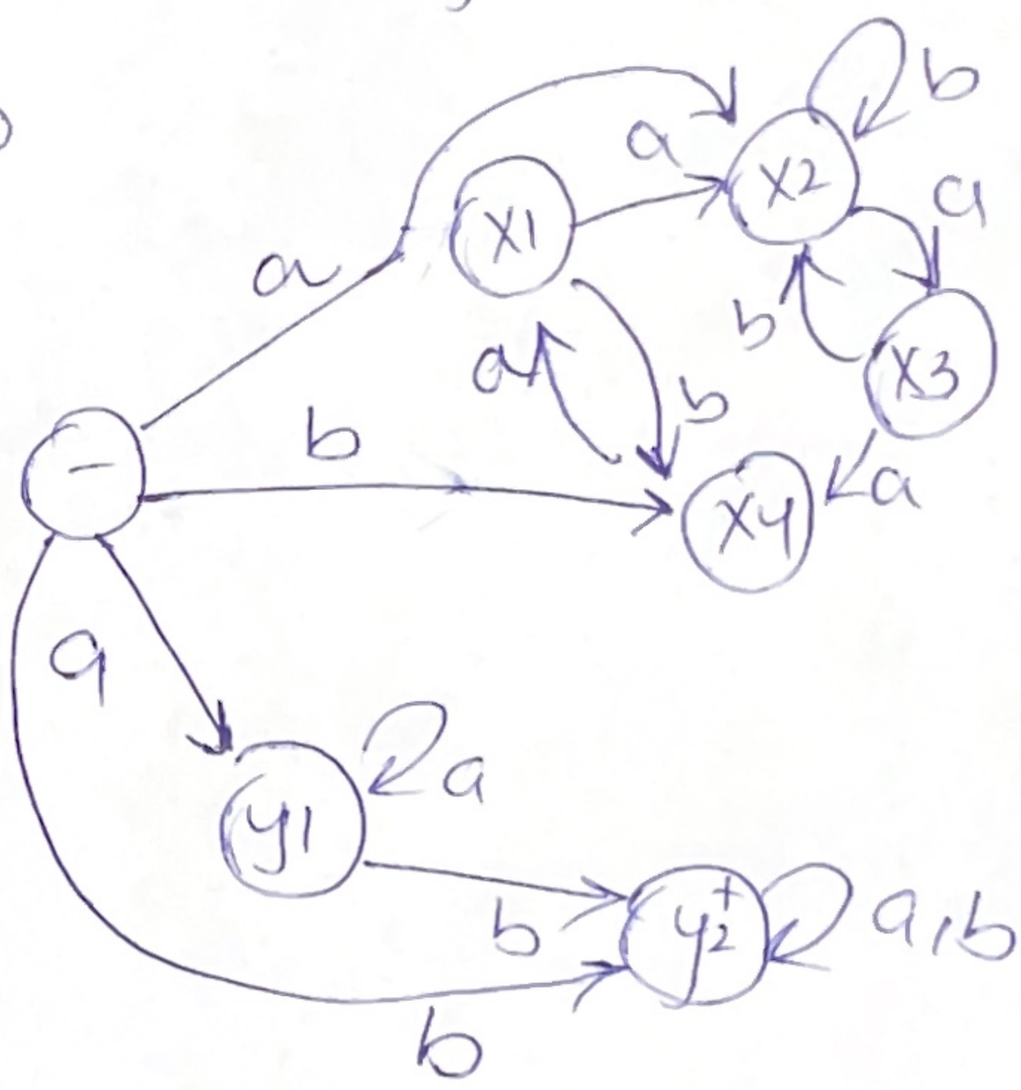
Example

x





To



x

THE END !

*